

TURBOQUANT PLUGIN SPECIFICATION

Dedicated Integration for DivTube Cockpit

Zero-GPU Vector Search • Local SEO Intelligence • Microsecond Latency

Document Type	Technical Plugin Specification
Version	1.0.0
Status	Draft for Implementation
Date	June 22, 2026
Target Platform	Steam Deck / Linux Handheld / Low-Power PC
Core Technology	TurboQuant.js (4-bit Quantized + Fast Hadamard Transform)

This document defines the complete specification for hooking TurboQuant as a first-class plugin into the DivTube Cockpit, enabling instant local semantic SEO analysis, title optimization, and vector-based content intelligence entirely offline.

1. EXECUTIVE SUMMARY

TurboQuant is a hyper-optimized, zero-GPU vector search engine built on 4-bit quantization and the Fast Hadamard Transform (FHT). When integrated as a dedicated plugin into DivTube Cockpit, it unlocks powerful AI-driven content optimization capabilities that run instantly and entirely locally on resource-constrained devices such as the Steam Deck.

Primary Value Proposition: Replace slow, cloud-dependent LLM calls and guesswork with mathematical, sub-millisecond vector similarity scoring against a registry of proven viral YouTube content. Content creators get precise, reproducible SEO guidance without ever leaving their local environment or uploading sensitive data.

Key Outcomes Delivered:

- Golden Niche SEO Reranker — Score any title/tag set against saved high-performing "Golden Curve" vectors (10M+ view videos).
- A/B Title Testing Simulator — Rank up to N title variants in microseconds using nearest-neighbor search.
- Vector Gap Analysis — Identify exactly which semantic clusters or tags are missing compared to a target viral video.
- Zero-Latency Local Clustering — Maintain and query a database of 100,000+ successful videos with microsecond latency and tiny memory footprint.

The plugin is implemented as a lightweight Node.js microservice (`turboquant_plugin.js`) that communicates with the existing Python-based DivTube Cockpit via simple IPC. All heavy lifting stays inside the highly optimized TurboQuant core (`turboquant.js`).

2. OBJECTIVES & SUCCESS CRITERIA

Objective	Success Metric
Enable instant local semantic scoring	Any title or tag set scored against Golden Curves in < 5 ms on Steam Deck hardware.
Eliminate cloud/LLM dependency for core SEO tasks	100% offline operation after initial Golden Curve registration.
Provide mathematically grounded recommendations	Inner-product similarity scores + ranked lists replace subjective "vibe checks".
Support large local registries	≥ 100k vector entries with < 50 MB RAM footprint and microsecond query times.
Seamless Cockpit integration	Native-feeling terminal commands (<code>/score</code> , <code>/test-title</code> , <code>/analyze-gaps</code>) with rich formatted output.
Privacy by design	No video metadata, titles, or analytics ever leave the user's device.

3. FEATURE SPECIFICATIONS

3.1 Golden Niche SEO Reranker

Users can register any high-performing YouTube video (or curated set) as a "Golden Curve" — a 4-bit quantized vector representing its semantic and stylistic signature. New titles, descriptions, or tag sets are vectorized on-the-fly and scored via inner-product similarity against one or more Golden Curves. The result is an immediate percentage match plus ranked recommendations.

Workflow:

1. `/register-golden <video_id>` or paste metadata → TurboQuant vectorizes & stores as named Golden Curve.
2. `/score-title "My Video Title Here" --against golden-curve-xyz` → Returns 0–100% match score + top contributing dimensions.
3. Optional: Score against multiple curves (niche A vs niche B) for positioning guidance.

3.2 A/B Title Testing Simulator

Provide 2–10 candidate titles for the same video. TurboQuant quantizes all candidates, performs fast nearest-neighbor lookup against the full registry of successful videos (or a chosen Golden Curve subset), and returns a ranked list from best to worst based on mathematical alignment. No LLM calls, no waiting, no API costs.

Example Command: `/test-titles --video my-video-123 "Title Option 1" "Title Option 2" "Title Option 3"`

Output: Ranked table with similarity scores, confidence, and quick rationale (top matching Golden Curves).

3.3 Missing Keyword & Tag Inference (Vector Gaps)

When a user's video vector is compared to a target Golden Curve, TurboQuant can surface the dimensional delta (the "gap"). DivTube's presentation layer then translates the largest-magnitude differing dimensions or residual clusters back into human-readable concepts or suggested tags (e.g., "Performance Optimization", "Beginner Friendly", "Speedrun").

Note: The core plugin returns raw delta vectors or top-k differing feature indices. The semantic label mapping can be implemented via (a) a lightweight static taxonomy maintained in DivTube, (b) pre-computed cluster centroids with human labels, or (c) an optional tiny local ONNX model if higher fidelity is required in later versions.

3.4 Zero-Latency Local Clustering & Search

Thanks to 4-bit packing and Fast Hadamard Transform acceleration, TurboQuant achieves extremely high query throughput with minimal memory. A full registry of 100,000 successful YouTube videos occupies only a few tens of megabytes and can be searched in microseconds on a Steam Deck CPU. This enables real-time clustering, duplicate detection, niche discovery, and "find similar successful videos" features without ever waking a discrete GPU or making a network request.

4. SYSTEM ARCHITECTURE & INTEGRATION

4.1 High-Level Components

Layer	Component	Responsibility
Data Ingestion	Java Engine (or equivalent)	Pull raw YouTube API metadata, analytics, thumbnails, transcripts as needed.
Cockpit UI	Python DivTube Cockpit	Terminal/dashboard interface; routes text to plugin; renders results; manages user workflows.
Plugin Core	Node.js microservice (turboquant_plugin.js)	Wraps turboquant.js; exposes CLI/HTTP IPC; manages Golden Curve registry & queries.
Vector Engine	turboquant.js (isomorphic)	Text vectorization, 4-bit quantization, FHT-accelerated inner-product & NN search.
Storage	Local binary registry files	4-bit packed vectors + metadata index. Simple file-based or lightweight embedded store.

4.2 Communication Pattern

The Python Cockpit spawns or connects to the Node.js microservice at startup (or on-demand). Communication uses lightweight IPC:

- Preferred (v1): stdio / child_process with JSON lines protocol (simple, no extra ports).
- Alternative: Local HTTP server on 127.0.0.1:random-port with fast JSON REST (Express/Fastify).
- Future: Unix domain socket or shared memory for even lower latency.

4.3 Data Flow Example (Score Title)

1. User types /score-title "New Video Title" in DivTube terminal.
2. Python parses command → sends JSON {action: "score", text: "New Video Title", curve: "golden-xyz"} to Node plugin via IPC.
3. turboquant_plugin.js calls turboquant.vectorize(text) → 4-bit quantize → inner_product against stored Golden Curve.
4. Plugin returns {score: 87.4, top_matches: [...], latency_ms: 0.42}.
5. Python renders formatted result in terminal with color-coded score, suggestions, and optional deep-dive.

5. TECHNICAL REQUIREMENTS

5.1 Runtime & Dependencies

Component	Requirement	Notes
Node.js	≥ 18.x LTS	For turboquant_plugin.js microservice
Python	≥ 3.10	Existing DivTube Cockpit runtime
turboquant.js	Latest (isomorphic)	Core vector engine; must expose vectorize, quantize, score, nn_search
IPC Library	None (stdlib) or fastify/express	stdio preferred for v1 simplicity
Optional	ONNX Runtime (Node)	Only if gap-to-tag translation uses tiny local model (future)

5.2 Target Hardware (Steam Deck Reference)

The plugin must deliver excellent interactive performance on AMD Ryzen Z1 Extreme / similar APUs with 16 GB RAM and integrated graphics:

- Single query (vectorize + score): < 2 ms typical, < 5 ms worst-case.
- NN search over 100k vectors: < 50 μs (microseconds) p50.
- Memory footprint (100k vectors @ 512-dim 4-bit): ~ 25–40 MB including index overhead.
- Cold start (load registry + plugin): < 800 ms.
- No GPU, no discrete graphics, no AVX-512 requirement (graceful fallback).

5.3 Storage & Persistence

Golden Curves and the main success registry are stored in a compact binary format (versioned header + 4-bit packed matrix + JSON metadata sidecar). The plugin provides import/export commands for sharing curated Golden Curve packs (e.g., "Gaming Niche 2025 Q2" pack). Atomic writes + checksums ensure registry integrity even on sudden power loss (important for handheld use).

6. COMMAND & API INTERFACE (v1)

All interaction happens through the DivTube terminal. The plugin exposes a small set of high-level commands. Under the hood these map to IPC calls.

Command	Description	Example
/register-golden	Vectorize & store a video or free-form text as a named Golden Curve.	/register-golden --name "Speedrun God" "https://youtu.be/..."
/list-curves	Show all saved Golden Curves with size and last-updated.	/list-curves
/score-title	Score a single title against one or more curves; return % match.	/score-title "My New Title" --curve speedrun-god
/test-titles	A/B test multiple titles; ranked output with scores.	/test-titles t1 "Title A" t2 "Title B" t3 "Title C"
/analyze-gaps	Compare user video vector vs target; surface missing semantic clusters.	/analyze-gaps --my-video abc123 --target speedrun-god

/search-similar	Find top-N most similar successful videos in the full registry.	/search-similar "My draft description..."
/export-pack	Export selected Golden Curves to portable .goldenpack file.	/export-pack speedrun-god fps-optimization

7. PERFORMANCE & SCALABILITY TARGETS

Metric	Target (Steam Deck)	Stretch Goal
Vectorize + 4-bit quantize (single text)	< 1.5 ms	< 0.8 ms
Inner-product score vs 1 Golden Curve	< 0.1 ms	< 50 µs
k-NN search (k=5) over 100k vectors	< 80 µs p50	< 30 µs p50
Full registry load time (100k entries)	< 600 ms	< 300 ms
Memory (100k × 512-dim @ 4-bit + index)	≤ 45 MB RSS	≤ 30 MB RSS
Plugin cold-start + IPC ready	< 1.2 s	< 700 ms

These targets are achievable because TurboQuant avoids the heavy matrix multiplications of typical vector DBs; the Fast Hadamard Transform reduces search to extremely cheap operations after the one-time quantization step.

8. SECURITY, PRIVACY & DATA GOVERNANCE

All processing is strictly local. No titles, tags, analytics, or video IDs are ever transmitted to any remote service. The only external data source is the public YouTube Data API (via the Java ingestion layer) when the user explicitly chooses to register a public video as a Golden Curve. Users retain full control: they can delete, export, or encrypt their local registry at any time.

- No telemetry or phoning home from the plugin.
- Optional: User can enable local-only logging (never uploaded).
- Golden Curve packs can be shared peer-to-peer; they contain only quantized vectors + human-readable labels the creator chose to include.
- Future: Registry file can be placed inside an encrypted container (LUKS/veracrypt) for users with strict OPSEC needs.

9. IMPLEMENTATION ROADMAP

Phase	Scope	Deliverables	Est. Effort
Phase 0	Foundation	turboquant_plugin.js skeleton, stdio IPC, basic vectorize/score round-trip, test harness on Steam Deck.	3–5 days
Phase 1	Core Features	Golden Curve CRUD, /score-title, /test-titles, persistent binary registry format.	7–10 days
Phase 2	Gap Analysis	/analyze-gaps returning delta vectors + basic cluster labels; integration into DivTube result renderer.	4–6 days
Phase 3	Polish & Scale	100k+ vector perf tuning, import/export packs, error handling, docs, packaging as DivTube plugin.	5–7 days
Phase 4	Advanced	Optional tiny local model for richer gap→tag translation; multi-curve comparison UI; community pack sharing workflow.	Future

10. RISKS, ASSUMPTIONS & MITIGATIONS

Risk / Assumption	Impact	Mitigation
turboquant.js embedding quality may not perfectly capture "viral vibe"	Medium — recommendations could feel off for some niches	Start with broad successful videos; allow users to curate their own Golden Curves; provide feedback loop for future fine-tuning of embedder if exposed.
Gap-to-English translation requires mapping layer	Low — core scoring still valuable	v1 returns raw deltas + top feature indices; simple static taxonomy in DivTube for common clusters; advanced model deferred.
Initial registry population effort	Medium — cold start feels empty	Ship with a small curated "starter pack" of public-domain successful video vectors; easy one-click import from YT URLs.
Node.js + Python IPC complexity	Low	stdio JSON-lines is battle-tested and simple; comprehensive error propagation and health-check commands.
4-bit quantization accuracy loss vs full-precision	Low — by design of TurboQuant	TurboQuant's FHT + training already accounts for this; inner-product ranking remains highly correlated with full-precision results.

Key Assumptions:

- turboquant.js already provides a stable, high-performance text→vector→quantized pipeline with the documented FHT optimizations.
- DivTube Cockpit already has a terminal command dispatcher that can be extended with plugin routes.
- YouTube Data API access (for ingestion) is already handled or can be added via the existing Java layer.

11. FUTURE EXTENSIONS (Post v1)

- Multi-modal vectors: combine title + thumbnail CLIP embedding + transcript snippets for richer Golden Curves.
- Real-time typing suggestions: stream partial title vectors as the user types in the DivTube editor.
- Community Golden Curve marketplace / pack sharing (with cryptographic signing for trust).
- Automatic niche clustering: discover emerging successful micro-niches from the local registry.
- Integration with DivTube analytics: auto-suggest Golden Curves based on the user's own historical top performers.
- ONNX / WebAssembly fallback for even broader device compatibility (including future Steam Deck OLED variants).

This specification is ready for implementation. All core capabilities are achievable with the described architecture and the unique performance characteristics of TurboQuant. The resulting plugin will give DivTube users a genuine competitive advantage: mathematically rigorous, private, instant SEO intelligence that runs beautifully on a handheld gaming PC.

— End of Specification v1.0 —